

Sistemas Operacionais

Implementação de comunicação Cliente/Servidor em C++

C++17

GRUPO	

Samuel Chad

Sarah Beatriz

Thiago Gonsalves

O que vamos ver

1

Linguagem e ambiente

C++17 · POSIX Sockets · Dev Container

2

Bibliotecas usadas

sys/socket · arpa/inet · thread + mutex

3

Comparação e Comportamento

Feito com threads vs Feito sem threads

4

Dificuldades

Desafios encontrados no desenvolvimento

5

Observações

Conclusões e aprendizados do grupo

LINGUAGEM E AMBIENTE

Linguagem

C++17

Compilador

g++ -std=c++17 -pthread

Threads

std::thread + pthread

Container

Dev Container · Ubuntu 24.04 | Docker Compose

Sockets

POSIX (sys/socket)

Headers e APIs utilizadas

<sys/socket.h>

socket() — cria o socket TCP

bind() — associa à porta

listen() — modo escuta

accept() — aceita conexão

send() / recv() — I/O

<arpa/inet.h>

sockaddr_in — estrutura de endereço

htons() — byte order de porta

inet_pton() — IP string → binário

inet_ntop() — binário → string

<thread> + <mutex>

std::thread — thread por cliente

std::mutex — proteção da lista

std::lock_guard — RAII lock

thread.detach() — sem join

COMPARAÇÃO E COMPORTAMENTO

✗ Sem Thread

Atende 1 cliente por vez

Segundo cliente fica bloqueado

Simples — sem race condition

Sem concorrência / sem overhead

Útil para aprendizado inicial

✓ Com Thread

N clientes simultâneos

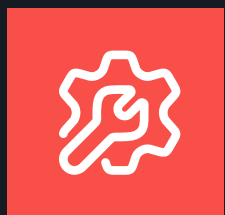
Cada cliente em thread própria

Mutex protege lista de clientes

Broadcast para todos os peers

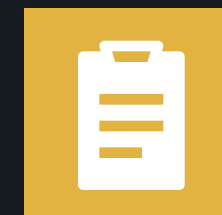
Necessário para chat real

Desafios encontrados



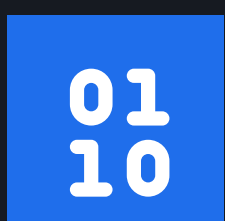
Configuração do Dev Container

Makefile não era detectado no Codespaces sem rebuild. Necessário reconstruir o container para aplicar o `postCreateCommand`.



Gerência de race conditions

Acesso concorrente ao vetor de clientes no servidor de chat exigiu uso de `std::mutex` e `lock_guard` para evitar corrupção de dados.



Byte order e portabilidade

Conversão entre host byte order e network byte order (`htons/ntohs`) é obrigatória — erro silencioso difícil de depurar.



Input concorrente no cliente

No cliente de chat, ler `stdin` e receber mensagens ao mesmo tempo exigiu duas threads separadas e `std::atomic<bool>` para sinalizar encerramento.

Conclusão

A API POSIX de sockets em C++ é verbosa, mas mapeada 1:1 com chamadas de sistema reais — o que torna o aprendizado valioso para entender como Python/Java abstraem esse nível.

`std::thread` + `std::mutex` é suficiente para o chat multi-cliente, mas em produção usaríamos `epoll` (Linux) ou `io_uring` para escalar sem criar uma thread por conexão.

O Dev Container garante ambiente reproduzível: qualquer membro do grupo roda exatamente o mesmo g++, bibliotecas e flags, sem surpresas de plataforma.

O flag `SO_REUSEADDR` é essencial durante desenvolvimento para evitar 'Address already in use' a cada restart do servidor.